

Object Oriented Programming with Java

What is 'Java'?

"Java is an object-oriented, high-level programming language."

- **Object-Oriented:** Based on real-world entities (Objects and Classes).
- **High-Level:** Easy to read & write, closer to human language than machine code.
- **Programming:** A way to give instructions to a computer.
- **Language:** A medium of communication.

*To make a computer/device perform a task, we need to **communicate** our ideas to it.*

*A **programming language** helps us give **instructions to the computer** in a structured way.*

*Java enables developers to do this in terms of **objects** and in a **language closer to natural language** easy to understand by the humans.*

Object Oriented Programming with Java

Java Source File Structure

First Program in Java

```
class HelloJava
{
    public static void main(String args[])
    {
        System.out.println("Hello, Java!");
    }
}
```

Java source code is written in a `.java` file.

Class Definition

- In Java, every program must be enclosed in a class.
- A class represents a set of similar objects.
- It is recommended that the class name matches the file name.
- The basic structure of a class looks like the following:

```
class HelloJava
{
    ...
}
```

Object Oriented Programming with Java

Java Source File Structure

Main Method

The main method is the starting point of every Java program. When we run a Java program, execution begins from the main method.

- Signature of the Main Method

The standard syntax of the main method is: `public static void main(String args[])`

<code>public</code> :	Makes the method accessible from anywhere.
<code>static</code> :	Allows the method to run without creating an object of the class.
<code>void</code> :	Means the method does not return any value
<code>main</code> :	This is the entry point of the Java program.
<code>String args[]</code> :	An array of Strings (named args in the example) is passed as parameter to main.

Object Oriented Programming with Java

Java Source File Structure

Main Method

- Definition or Body of the Main Method

The body of the main method contains the actual instructions to be executed. Statements inside the method are written inside curly braces {}.

```
public static void main(String args[])  
{  
    System.out.println("Hello, Java!");  
}
```

- `System.out.println("Hello, Java!");`
prints text on the screen.
- Every statement in Java ends with a semicolon (;).

Object Oriented Programming with Java

Compiling and Executing a Java Program

Compiling the Java Program

- Command to compile the Java program:
javac HelloJava.java
- *javac* is the Java compiler that converts Java source code into **bytecode**.
- If there are no errors, it generates a file: **HelloJava.class** .
- This .class file contains bytecode, which is not machine-specific but can be executed on any platform with a JVM.

Executing the Java Program

- Command to execute:
java HelloJava
- *java* is the Java interpreter that runs the compiled bytecode.
- *Output: Hello, Java!*

Object Oriented Programming with Java

Java Environment

JDK (Java Development Kit)

The JDK is a complete software development package that includes everything needed to *develop, compile, and run Java applications*. It contains the *JRE (Java Runtime Environment) and development tools* like the Java compiler (javac), debugger, and other utilities. Without the JDK, we cannot develop and compile Java programs.

JRE (Java Runtime Environment)

The JRE is required to *run Java applications*. It includes the *JVM (Java Virtual Machine) along with core Java libraries and runtime resources*. While the JRE allows you to execute Java programs, it does not include the compiler or development tools, so you cannot develop Java applications with just the JRE.

JVM (Java Virtual Machine)

The JVM is a key component that *makes Java platform-independent*. It *converts Java bytecode (.class files) into machine code* so that the program can run on any operating system. JRE contains a JVM, ensuring that Java programs can run on different devices without modification.

JDK $\supset$ JRE $\supset$ JVM

JDK contains JRE, and JRE contains JVM.

```
class First
{
    public static void main(String args[])
    {
        System.out.println("My First Program.");
    }
}
```

Object Oriented Programming with Java

Data Types

“**Data types** specify which type of value a variable can hold in a programming language.”

There are two types of data types:

Primitive Data Types:

These are the basic data types in Java, representing single values like numbers, characters, and boolean values. They are predefined by the language and include types such as int, char, boolean, etc. They are also commonly referred to as simple types.

Non-Primitive Data Types:

These are more complex data types in Java, such as objects and arrays, which are derived from primitive types and can hold multiple values or more intricate data structures. They are also called reference types.

Object Oriented Programming with Java

Data Types

Primitive Data Types:

Java defines eight primitive types of data: *byte, short, int, long, char, float, double, and boolean*.

These data types can be put in four groups:

1. Integers

This group includes *byte, short, int, and long*, which are for whole-valued signed numbers.

2. Floating-point numbers

This group includes *float* and *double*, which represent numbers with fractional precision.

3. Characters

This group includes *char*, which represents symbols in a character set, like letters and numbers.

4. Boolean

This group includes *boolean*, which is a special type for representing true/false values.

Object Oriented Programming with Java

Data Types

Primitive Data Types:

1. Integer Types

- Java provides four integer data types: **byte**, **short**, **int**, and **long**.
- All of these support both positive and negative values (signed integers).
- Unlike some other programming languages, Java does not include support for unsigned integers, which store only non-negative values.
- The width and ranges of the integer types are given below:

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

Object Oriented Programming with Java

Data Types

Primitive Data Types:

1. Integer Types

- **byte**: The byte is the smallest integer type which is useful when dealing with raw binary data. It is a signed 8-bit type that has a range from -128 to 127.
- **short**: short is a signed 16-bit type. It has a range from -32,768 to 32,767. It is the least used Java type.
- **int**: The int type is the most commonly used integer type. It is a signed 32-bit type that has a range from -2,147,483,648 to 2,147,483,647. int is often the best choice when an integer is needed as even when byte and short values are used in an expression, they are promoted to int when the expression is evaluated.
- **long**: The long type is a 64-bit signed integer, suitable for situations where an int cannot accommodate the required value. Its extensive range makes it ideal for handling large whole numbers.

Object Oriented Programming with Java

Data Types

Primitive Data Types:

1. Integers

Example

```
public class IntegerTypesDemo {  
    public static void main(String[] args) {  
        // 1. byte: Smallest integer type, range: -128 to 127  
        byte smallValue = 100;  
        System.out.println("Byte value: " + smallValue);  
  
        // 2. short: Larger than byte, range: -32,768 to 32,767  
        short mediumValue = 30000;  
        System.out.println("Short value: " + mediumValue);  
  
        // 3. int: Default integer type, range: -2,147,483,648 to 2,147,483,647  
        int normalValue = 1_000_000; // Underscores improve readability  
        System.out.println("Int value: " + normalValue);  
  
        // 4. long: For very large numbers,  
        // range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807  
        long largeValue = 9_000_000_000L; // 'L' suffix is needed for long literals  
        System.out.println("Long value: " + largeValue);  
    }  
}
```

Object Oriented Programming with Java

Data Types

Primitive Data Types:

2. Floating-point Types

- Java provides two floating-point data types: **double**, and **float**.
- Floating-point numbers, also known as real numbers, are used when evaluating expressions that require fractional precision.
- float and double, represent single and double precision numbers, respectively.
- The width and ranges of the integer types are given below:

Name	Width	Range
double	64	4.9e−324 to 1.8e+308
float	32	1.4e−045 to 3.4e+038

Object Oriented Programming with Java

Data Types

Primitive Data Types:

2. Floating-point Types

- **float:** The float type is a 32-bit single-precision floating-point number. It can represent numbers with fractional precision and has a range from $1.4\text{e-}45$ to $3.4\text{e+}38$. It is generally used when memory efficiency is a concern and when high precision is not required.
- **double:** The double type is a 64-bit double-precision floating-point number. It has a much larger range and higher precision compared to float, from $4.9\text{e-}324$ to $1.8\text{e+}308$. It is the default choice for representing decimal values in Java and is widely used for scientific and mathematical computations.

Object Oriented Programming with Java

Data Types

Primitive Data Types:

2. Floating-point Types

Example:

```
public class FloatingPointTypesDemo {  
    public static void main(String[] args) {  
        // Declaring and initializing float and double variables  
  
        float floatValue = 1.123456789f; // 'f' is added to indicate it's a float value  
        double doubleValue = 1.123456789123456; // double has higher precision  
  
        System.out.println("Float value: " + floatValue);  
        System.out.println("Double value: " + doubleValue);  
    }  
}
```

Object Oriented Programming with Java

Data Types

Primitive Data Types:

3. Character Type

- In Java, **char** is used to store characters.
- Java uses **Unicode** to represent a wide range of characters from various languages. It ensures global portability
- A char is **16 bits** wide, unlike C/C++ where char is typically 8 bits.
- The **range** of a char is from **0 to 65,536**. char values in Java cannot be negative.
- Though Java uses Unicode for broader compatibility but ASCII (0-127) and ISO-Latin-1 (0-255) character sets are also supported.

Object Oriented Programming with Java

Data Types

Primitive Data Types:

3. Character Type

Example:

```
class CharTypeDemo {  
    public static void main(String args[]) {  
        char ch1, ch2;  
  
        // Assigning numeric Unicode values  
        ch1 = 65;        // ASCII code for 'A'  
        ch2 = 'B';        // Character 'B'  
  
        // Displaying the characters  
        System.out.print("ch1, and ch2: ");  
        System.out.println(ch1 + " " + ch2);  
    }  
}
```

Object Oriented Programming with Java

Data Types

Primitive Data Types:

4. Boolean Type

- Java has a primitive type, called **boolean**, which can have only one of two possible values, **true** or **false**.
- This is the type returned by all relational operators.
- boolean is also the type required by the conditional expressions that govern the control statements such as if.

Object Oriented Programming with Java

Data Types

Primitive Data Types:

4. Boolean Type

Example:

```
class BooleanTypeDemo {
    public static void main(String args[]) {
        boolean b;
        b = false;
        System.out.println("b is " + b);

        b = true;
        // a boolean value can control the if statement
        if (b)
            System.out.println("Inside if.");

        // outcome of a relational operator is a boolean value
        System.out.println((8 > 9));
    }
}
```

```
class IntegerTypesDemo
{
    public static void main(String args[])
    {
        byte byteVar = 111;
        System.out.println("BYTE VALUE: "+byteVar);

        short shortVar = 31212;
        System.out.println("SHORT VALUE: "+shortVar);

        int intVar = 1_000_222;
        System.out.println("INT VALUE: "+intVar);

        long longVar = 9_222_111_333L;
        System.out.println("LONG VALUE: "+longVar);

        float floatVar = 3.14F;
        System.out.println("FLOAT VALUE: "+floatVar);

        double doubleVar = 3.1415926535;
        System.out.println("DOUBLE VALUE: "+doubleVar);

        char charVar1 = 'C';
        char charVar2 = 68;
        charVar2++; //charVar2+1

        System.out.println("CHAR VALUE 1: "+charVar1);
        System.out.println("CHAR VALUE 2: "+charVar2);

        boolean boolVar=true;
        if(boolVar)
```

```
{  
    System.out.println("BOOLEAN VALUE : "+boolVar);  
    System.out.println(4>5);  
}  
  
}  
  
}
```

Object Oriented Programming with Java

Variables

*“A **variable** is a named memory location used to store data that can be modified during program execution.”*

- The variable is the **basic unit of storage** in a Java program.
- A variable is defined by the combination of a **type**, an **identifier**, and an optional **initial value**.
- All variables have a **scope**, which defines their **visibility**, and a **lifetime**.

Declaration

The variables are declared as follows:

```
int var;
```

```
int var=10;
```

Here, **int** is the **data type**, **var** is the **identifier**, and **10** is the **initial value**.

We may declare more than one variables of the same type in one statement:

```
int var1=20,var2,var3;
```

Object Oriented Programming with Java

Variables

Dynamic Initialization

A variable may be initialized dynamically, using any expression valid at the time the variable is declared.

```
int var1=20,var2=var1+30,var3=var1+var2;
```

Scope and Lifetime

A block, beginning with an opening curly brace and ending by a closing curly brace, defines a **scope**.

- A scope determines what variables are **visible** to other parts of the program
- Scope also defines the **lifetime** of these variables. Variables are created when their scope is entered and destroyed when their scope is left.
- The **method scope begins with its opening curly brace** but method parameters are included in the method's scope.
- Variable declared within a scope are not visible to the code outside that scope. **Scopes can be nested**. The variables declared in the outer scope are visible to the inner scope but the reverse is not true.

Object Oriented Programming with Java

Variables

Example:

```
public class VarDemo {  
    public static void main(String[] args) {  
        int var=10;  
        int var1=20,var2=var1+30,var3=var1+var2;  
        System.out.println(var);  
        System.out.println(var1+var2+var3);  
  
        if(var3>var)  
        {  
            int var4=100;  
            System.out.println(var3+" is greater than "+ var);  
        }  
        else  
        {  
            int var5=200;  
            System.out.println(var+" is greater than "+ var3);  
        }  
        System.out.println(var4+var5);// Error  
    }  
}
```

Object Oriented Programming with Java

Control Flow

*“**Control flow** determines the order of execution of statements in a Java program.”*

Java provides various **control flow mechanisms** to execute code

- sequentially,
- selectively,
- iteratively, or by
- jumping to different parts of the program.

Types of Control Flow

Linear (Sequential) Control Flow

The default execution order where statements are executed one after another in the order they appear.

Example:

```
System.out.println("Start");  
int x = 10;  
System.out.println("Value of x: " + x);  
System.out.println("End");
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

It changes the normal sequence of execution. It includes Selection, Iteration, and Jump Statements.

1. Selection (Conditional) Statements

Selection Statements are used to execute a block of code based on conditions. The different selection statements in Java are ‘if’ (or if-else, if-else-if) and ‘switch’ statements.

‘if’ Statement

The if statement helps make decisions in a program. It checks a condition and runs code only if the condition is true.

- There may be an optional ‘else’ part with an if.
- Without {}, only the first line after if or else runs.

Example:

```
if (age >= 18) {  
    System.out.println("You can vote!");  
}
```

Example:

```
if (age >= 18)  
    System.out.println("You can vote!");  
else  
    System.out.println("You can not vote!");
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

1. Selection (Conditional) Statements

'if' Statement

- The if-else-if ladder checks multiple conditions in order.

Example:

```
if (marks >= 90) {  
    System.out.println("Grade: A");  
}  
else if (marks >= 75) {  
    System.out.println("Grade: B");  
}  
else {  
    System.out.println("Grade: C");  
}
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

1. Selection (Conditional) Statements

'switch' Statement

The switch statement is used to simplify decision-making when multiple conditions need to be checked. It provides a way to execute different parts of the code based on the value of an expression/variable.

```
switch (expression) {  
  
    case value1:  
        // Code to execute if  
        expression matches value1  
        break;  
  
    case value2:  
        // Code to execute if  
        expression matches value2  
        break;  
  
    default:  
        // Code to execute if  
        no case matches  
}
```

- It compares a variable/ expression against **multiple possible values** and executes the matching case.
- Each case should end with **break** (otherwise, the program will continue checking the next case even if a match is found).
- The **default** case is optional but executes when no match is found.
- It works well when **replacing multiple if-else-if** conditions.

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

1. Selection (Conditional) Statements

'switch' Statement

Example: 

```
int day = 8;
switch (day) {
    case 1:
        System.out.println("Sunday");
        break;
    case 2:
        System.out.println("Monday");
        break;
    case 3:
        System.out.println("Tuesday");
        break;
    case 4:
        System.out.println("Wednesday");
        break;
```

```
        case 5:
            System.out.println("Thursday");
            break;
        case 6:
            System.out.println("Friday");
            break;
        case 7:
            System.out.println("Saturday");
            break;
        default:
            System.out.println("Invalid day!");
    }
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

1. Selection (Conditional) Statements

'switch' Statement

Example: 

```
String month = "OCT";
switch (month) {
    case "JUL":
    case "AUG":
    case "SEP":
    case "OCT":
    case "NOV":
    case "DEC":
        System.out.println("ODD Semester");
        break;
    case "JAN":
    case "FEB":
    case "MAR":
    case "APR":
    case "MAY":
```

```
        case "JUN":
            System.out.println("EVEN Semester");
            break;
        default:
            System.out.println("Invalid Month!");
            break;
    }
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

It changes the normal sequence of execution. It includes Selection, Iteration, and Jump Statements.

2. Iteration (Looping) Statements

An iteration statement is used to create loop which repeatedly executes the same set of instructions until a termination condition is met. Java's iteration statements are **for**, **while**, and **do-while**.

'while' Loop

The **'while'** loop repeats a statement or block while its controlling condition is true. When condition becomes false, control passes to the next line of code immediately following the loop.

- Without {}, only the first line after while runs.
- Since the while loop evaluates its conditional expression at the top of the loop, the body of the loop will not execute even once if the condition is false.
- The body of the while can be empty.

Example:

```
int n=1;

while(n<=10)
{
    System.out.println(n);
    n++;
}
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

2. Iteration (Looping) Statements

'do-while' Loop

The **do-while** loop always executes its body at least once as it tests the termination condition at the end of the loop rather than at the beginning.

Example:

```
int n=1;

do
{
    System.out.println(n);
    n++;
} while(n<=10);
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

2. Iteration (Looping) Statements

'for' Loop

The **for** loop in Java is used to repeat a set of statements a fixed number of times. The new **for-each** loop helps iterate through arrays and collections easily.

1. Traditional **for** Loop:

It is used when the number of iterations is known. It consists of three parts:

Initialization:	Sets the loop control variable.
Condition:	Checks if the loop should continue.
Counter:	Modifies the loop counter variable after each iteration.

Example:

```
for (int i = 1; i <= 10; i++)  
    System.out.println(i);
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

2. Iteration (Looping) Statements

'for' Loop

2. **for-each** Loop:

It is used to iterate over arrays or collections without needing an index.

- It is simpler and safer (avoids index errors).
- It is best suited for reading elements but not modifying them.

Example:

```
int[] numbers = { 10, 20, 30, 40 };  
for (int num : numbers) {  
    System.out.println(num);  
}
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

It changes the normal sequence of execution. It includes Selection, Iteration, and Jump Statements.

3. Jump Statements

Jump statements in Java are used to transfer control to a different part of the program.

'break' Statement

The break statement is used to immediately exit a loop or a switch statement, skipping any remaining iterations or cases.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3)  
        break; // Exits the loop when i is 3  
    System.out.println(i);  
}
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

3. *Jump Statements*

'continue' Statement

The continue statement is used to skip the current iteration of a loop and move to the next iteration without executing the remaining statements in the loop body.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3)  
        continue; // Skips the iteration when i is 3  
    System.out.println(i);  
}
```

Object Oriented Programming with Java

Control Flow

Types of Control Flow

Non-Linear Control Flow

3. *Jump Statements*

'return' Statement

The return statement is used to exit from a method and optionally return a value to the calling method.

Example:

```
int add(int a, int b) {  
    return a + b; // Returns the sum of a and b  
}
```

```
class VariablesDemo
{
    /* This is the program to demonstrate
    the variables in Java*/
    public static void main(String args[])
    {
        int var=23; //Variable Declaration
        int var1 = var *4; //Dynamic Initialization
        System.out.println(var1);
        if (var<34)
        {
            int var2=56;
            System.out.println(var2);
            System.out.println(var);
        }
        System.out.println(var2);
    }
}
```

Object Oriented Programming with Java

Operators

*“An **operator** in Java is a symbol or keyword used to perform operations on variables and values.”*

Example:

```
int a = 10, b = 5;  
int sum = a + b; // '+' is an operator that adds two numbers  
System.out.println(sum);
```

Types of Operators

- Arithmetic Operators
 - Unary Arithmetic Operators
 - Binary Arithmetic Operators
- Relational (Comparison) Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Conditional Operator (Ternary)

Object Oriented Programming with Java

Operators

Arithmetic Operators

These operators are used to perform basic mathematical operations such as addition, subtraction, multiplication, division, and modulus.

Binary Arithmetic Operators: These operate on two operands.

Examples: + (addition), - (subtraction), * (multiplication), / (division), and % (modulus for remainder calculation).

Operator	Description	Example [int a=5,b=3;]	Result
+	Addition	a + b	8
-	Subtraction	a – b	2
*	Multiplication	a * b	15
/	Division (quotient)	a / b	1
%	Modulus (remainder)	a % b	2

Object Oriented Programming with Java

Operators

Arithmetic Operators

Unary Arithmetic Operators: These operate on a single operand.

Examples: + (unary plus), - (unary minus), ++ (increment), -- (decrement).

Operator	Description	Example	Result
+	Unary plus (no effect, used for clarity)	int a = +5;	a = 5
-	Unary minus (negates a value)	int a = -5;	a = -5
++	Increment (increases value by 1)	int a = 5; a++;	a = 6
--	Decrement (decreases value by 1)	int a = 5; a--;	a = 4

Object Oriented Programming with Java

Operators

Relational Operators

These operators compare two values and return a boolean result (true or false). They are commonly used in decision-making.

Examples: == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), <= (less than or equal to).

Operator	Description	Example [int a=5, b=3;]	Result
==	Equal to	a == b	FALSE
!=	Not equal to	a != b	TRUE
>	Greater than	a > b	TRUE
<	Less than	a < b	FALSE
>=	Greater than or equal to	a >= b	TRUE
<=	Less than or equal to	a <= b	FALSE

Object Oriented Programming with Java

Operators

Logical Operators

These operators are used to combine multiple boolean expressions and return a boolean result. They are mainly used in conditional statements.

Examples: && (logical AND – returns true if both conditions are true), || (logical OR – returns true if at least one condition is true), ! (logical NOT – reverses the boolean value).

Operator	Description	Example [int a=5, b=3;] [int c=3, d=1;]	Result
&&	Logical AND (both conditions must be true)	(a > b) && (c > d)	TRUE
	Logical OR (at least one condition must be true)	(a < b) (c > d)	TRUE
!	Logical NOT (reverses boolean value)	!(a > b)	FALSE

Object Oriented Programming with Java

Operators

Bitwise Operators

These operators perform operations at the binary (bit) level, modifying individual bits in an integer.

Examples: & (Bitwise AND), | (Bitwise OR), ^ (Bitwise XOR), ~ (Bitwise Complement), << (Left Shift), >> (Right Shift), and >>> (Unsigned Right Shift).

Operator	Description	Example [int a=5, b=3;] [int c=8, d=1;]	Result
&	Bitwise AND	a & b (0101 & 0011)	1 (0001)
	Bitwise OR	a b (0101 0011)	7 (0111)
^	Bitwise XOR	a ^ b (0101 ^ 0011)	6 (0110)
~	Bitwise Complement	~a (~0101)	-6
<<	Left shift (multiplies by 2)	c << d	16
>>	Right shift (divides by 2)	c >> d	4
>>>	Unsigned right shift	-c >>> d	2147483644

Object Oriented Programming with Java

Operators

Assignment Operators

These operators assign values to variables and also provide shorthand expressions to modify values.

Examples: = (simple assignment), += (addition with assignment), -= (subtraction with assignment), *= (multiplication with assignment), /= (division with assignment), %= (modulus with assignment).

Operator	Description	Example [int a;]	Result
=	Assign value	a = 10;	10 (a = 10)
+=	Add and assign	a += 5;	15 (a = a + 5)
-=	Subtract and assign	a -= 3;	12 (a = a – 3)
*=	Multiply and assign	a *= 2;	24 (a = a * 2)
/=	Divide and assign	a /= 2;	12 (a = a / 2)
%=	Modulus and assign	a %= 2;	0 (a = a % 2)

Object Oriented Programming with Java

Operators

Conditional Operator

This is a shorthand operator for if-else statements, allowing conditional expressions to be evaluated in a single line.

Syntax: condition ? value_if_true : value_if_false

(If the condition is true, it returns value_if_true; otherwise, it returns value_if_false).

Operator	Description	Example [int a=5, b=4;]	Result
?:	Shorthand for if-else	int min = (a < b) ? a : b;	4

```
class OperatorsDemo
{
    public static void main(String args[])
    {
        int a=15, b=8;

        System.out.println(a+b);
        System.out.println(a-b);
        System.out.println(a*b);
        System.out.println(a/b);
        System.out.println(a%b);

        System.out.println(+a);
        System.out.println(-a);

        System.out.println(++a);
        System.out.println(--a);

        System.out.println(a==b); //False
        System.out.println(a!=b); //True
        System.out.println(a>b); //True
        System.out.println(a<b); //False
        System.out.println(a>=b); //True
        System.out.println(a<=b); //False

        System.out.println((a>b)&&(b>5));
        System.out.println((a>b) || (b<5));
        System.out.println(!(a>b));

        a=5;
        b=3;
```

```
int c=8;
int d=1;
System.out.println(a&b);
System.out.println(a|b);
System.out.println(a^b);
System.out.println(~a);
```

```
System.out.println(c<<d);
System.out.println(c>>d);
System.out.println(~c>>>d);
```

```
c+=2; //c=c+2
System.out.println(c);
c-=2; //c=c-2
System.out.println(c);
c*=2; //c=c*2
System.out.println(c);
c/=2; //c=c/2
System.out.println(c);
c%=2; //c=c%2
System.out.println(c);
```

```
int result;
result = a<b?b-a:a-b;
System.out.println(result);
```

```
}
```


Object Oriented Programming with Java

Object Oriented Programming

“Programming Oriented towards Objects.”

OBJECT

*“An object is a **real-world entity** that has a distinct **existence** (physical or conceptual), possesses **attributes**, exhibits well-defined **behavior** (actions), and **interacts** with other objects within its environment.”*

- *Real-world entity* – Objects represent things we observe in reality.
- *Distinct existence (physical or conceptual)* – Object can be tangible (e.g., a car) or intangible (e.g., a bank account)
- *Attributes* – Object has identifiable characteristics that define the properties/data of an object.
- *Behavior (actions)* – Objects perform actions.
- *Interaction with other objects* – Objects don't exist in isolation but communicate with each other.

Object Oriented Programming with Java

Object Oriented Programming

“Programming Oriented towards Objects.”

OBJECT

Examples:

Object	Attributes (Properties)	Behaviors (Actions)
Car	Color, Model, Speed	Accelerate, Brake, Blow horn
Student	Name, Roll No., Age	Study, Give Exams, Attend Class
Mobile Phone	Brand, Model, Battery Level	Call, Message, Browse Internet

Object Oriented Programming with Java

Object Oriented Programming

CLASS

The Problem:

Defining Each Object Separately is Complex

- Every object has attributes and behavior.
- If we define each object individually, it becomes repetitive and hard to manage as a group of objects share common characteristics (like all students have attributes: rollNo, name, and the behavior: give exams, attend classes etc.).
- Example: If we create 100 students, defining each one separately is inefficient.

The Solution:

Use a Class

- Instead of defining each object separately, we define a common structure once.
- This structure is called a Class.
- Objects are then created based on this Class.

Object Oriented Programming with Java

Object Oriented Programming

Class

*“A Class is a **blueprint/ template** that defines the **common attributes and behavior** shared by all objects of that type.”*

- *A class represents a set of similar objects.*
- *Objects created from a class follow the structure defined in it.*

Example:

Class: Student

This class represents a general definition of a student.

- **Attributes (Properties):**
Name, Age, Roll Number, Course, Marks
- **Behaviors (Actions):**
Study, Take Exams, Submit Assignments,
Attend Classes

Object Oriented Programming with Java

Object Oriented Programming

Defining a Class in Java

The keyword *class* is used to define a class.

- *Variables* are used to define attributes that store object data.
- *Methods* are used to define behavior that represent the actions the objects may perform.

Syntax:

```
class ClassName {  
    // Attributes (Data Members)  
    DataType attribute1;  
    DataType attribute2;  
  
    // Behavior (Member Methods)  
    return_type method1(parameters) {  
        // Code for method1  
    }  
    return_type method2(parameters) {  
        // Code for method2  
    }  
}
```

Object Oriented Programming with Java

Object Oriented Programming

Defining a Class in Java

Example:

```
class Student {  
    // Attributes (Data members)  
    String name;  
    int age;  
    int rollNumber;  
  
    // Behavior (Member Methods)  
    void study() {  
        System.out.println(name + " is studying.");  
    }  
  
    void takeExam() {  
        System.out.println(name + " is taking an exam.");  
    }  
}
```

Object Oriented Programming with Java

Object Oriented Programming

Creating Objects of a Class

Objects are created from a class in the following way:

```
ClassName objectName;  
objectName = new ClassName();
```

OR

```
ClassName objectName = new ClassName();
```

ClassName → Name of the class.

objectName → Reference variable that stores the object.

new → Allocates memory for the object.

ClassName() → Calls the class constructor to initialize the object.

Example:

```
Student stuObj;  
stuObj = new Student();
```

OR

```
Student stuObj = new Student();
```

Object Oriented Programming with Java

Object Oriented Programming

Assigning Values to Object Attributes

We use the dot (.) operator to access attributes and assign values.

```
stuObj.name = "Mayank";  
stuObj.age = 19;
```

Calling Methods on the Object

The dot (.) operator is used to access methods as well.

```
stuObj.display();
```

Each object has its own copy of attributes but follows the same structure.

Object Oriented Programming with Java

Object Oriented Programming

Example:

```
class Student {  
    String name;  
    int age;  
    void display() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.name = "John";  
        s1.age = 20;  
        s1.display();  
    }  
}
```

```
class Student
{
    String name;
    int rollNo;
    int age;

    void study()
    {
        System.out.println(name+" is studying.");
    }

    void takeExam()
    {
        System.out.println(name+" is taking exam.");
    }
}
```

```
class MainClass
{
    public static void main(String args[])
    {
        Student obj1;
        obj1= new Student();

        obj1.name="Saurabh";
        obj1.rollNo=102;
        obj1.age = 19;

        obj1.study();
        obj1.takeExam();
    }
}
```

```
Student obj2 = new Student();
```

```
obj2.name="Mayank";
```

```
obj2.rollNo=101;
```

```
obj2.age=19;
```

```
obj2.study();
```

```
obj2.takeExam();
```

```
}
```

```
}
```

Object Oriented Programming with Java

Class

Methods

*“A **method** is a block of code that performs a specific task.”*

- *It helps in code **reusability** and **modularity**.*
- *Methods are used to define **behavior** of an object.*

➤ *Defining a method*

The method in Java is defined as per the following syntax:

```
returnType methodName(parameters) {  
    // Method body  
    return value; // (if returnType is not void)  
}
```

Object Oriented Programming with Java

Class

Methods

Example:

```
class Example {  
    // Method with no parameters and void return type  
    void display()  
    {  
        System.out.println("Java");  
    }  
  
    // Method with parameters and a return type  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

Object Oriented Programming with Java

Class

Methods accessing data members in a class

“Methods inside a class can access its data members (variables) to perform operations.”

- *Instance variables* belong to an object of the class.
- They can be accessed directly within *instance methods* of the same class.

Example:

```
class Calculator {  
  
    //instance variables  
    int operand1;  
    int operand2;  
  
    // Method accessing instance variables  
    int add() {  
        return operand1 + operand2;  
    }  
}
```

U-1 Lec-6 Code-1

```
class MyClass
{

    int operand1;
    int operand2;

    int add()
    {
        return operand1 + operand2;
    }
}

class MainMethodClass
{
    public static void main(String args[])
    {
        MyClass obj = new MyClass();
        obj.operand1 = 23;
        obj.operand2 = 12;
        System.out.println("Result: "+obj.add());
    }
}
```

U-1 Lec-6 Code-2

```
class Rectangle
{

    int length;

    int width;

    int area()
```

```
        {  
            return length * width;  
        }  
    }  
  
    class RectangleMain  
    {  
        public static void main(String args[])  
        {  
            Rectangle r1 = new Rectangle();  
            Rectangle r2 = new Rectangle();  
            Rectangle r3 = new Rectangle();  
  
            r1.length = 2;  
            r1.width = 3;  
  
            r2.length = 3;  
            r2.width = 3;  
  
            r3.length = 5;  
            r3.width = 1;  
  
            System.out.println("Area of r1: "+r1.area());  
            System.out.println("Area of r2: "+r2.area());  
            System.out.println("Area of r3: "+r3.area());  
        }  
    }
```

Object Oriented Programming with Java

Abstraction

*“**Abstraction** is the process of exposing only the essential and relevant details of an object while hiding the unnecessary and irrelevant details.”*

Example:

When you use an **ATM**, you perform actions like inserting a card, entering a PIN, withdrawing cash, and checking balance.

However, you don't need to know how the bank processes transactions, verifies your PIN, or connects to banking servers.

- Abstraction makes things **simple to understand** by hiding complex details.
- One has to focus only on **what an object does**, and **not how it works internally**.

Object Oriented Programming with Java

Encapsulation

*“**Encapsulation** is the process of wrapping attributes (data members/instance variables) and behavior (methods) together into a single unit ”*

Example:

A **capsule in medicine** is a real-life example of encapsulation. The medicine (data) inside the capsule (class) is protected and can only be consumed in a controlled manner.

- Encapsulation ensures **data security** by restricting direct access to critical data.
- It provides **controlled access** to attributes through methods.

Object Oriented Programming with Java

Abstraction and Encapsulation: Complementary Concepts

“Encapsulation enables hiding and exposing the details told by the abstraction to be hidden and exposed.”

- **Abstraction decides** what details should be hidden and what should be shown.
- **Encapsulation implements this decision** by restricting access to certain details while allowing others.

“What in an encapsulation unit is to be hidden and what to be exposed is told by Abstraction.”

- Encapsulation bundles data (attributes) and behavior (methods) together in a unit (class).
- Abstraction determines which parts of the unit should be accessible and which should remain hidden.

Object Oriented Programming with Java

Access Specifiers

“Access specifiers in Java control the visibility of data members (attributes) and methods (behavior) within a class.”

- They help implement Encapsulation by restricting or allowing access to certain parts of a class.

Types of Access Specifiers

private	Accessible only within the same class (Used for sensitive data).
public	Accessible from anywhere (Used for important methods and attributes).
protected	Accessible within the same package and by subclasses (within as well as outside the package).
default	Accessible within the same package (If no access specifier is mentioned, it is considered default).

Object Oriented Programming with Java

Access Specifiers

Example:

```
class Rectangle {
    private int length;
    private int width;

    public int area() {
        return length * width;
    }
}

class RectangleMain {
    public static void main(String args[]) {
        Rectangle r1 = new Rectangle();

        // Private data members are not allowed to be accessed outside the class.
        r1.length = 2;
        r1.width = 3;

        // Public member method is allowed to be accessed outside the class.
        System.out.println("Area of r1: "+r1.area());
    }
}
```

How will these
attributes get
the values?

```
class Rectangle
{
    //public int length;
    //public int width;

    private int length;
    private int width;

    int area()
    {
        return length * width;
    }
}

class RectangleMain
{
    public static void main(String args[])
    {
        Rectangle r1 = new Rectangle();
        Rectangle r2 = new Rectangle();
        Rectangle r3 = new Rectangle();

        r1.length = 2;
        r1.width = 3;

        r2.length = 3;
        r2.width = 3;

        r3.length = 5;
        r3.width = 1;

        System.out.println("Area of r1: "+r1.area());
    }
}
```

```
System.out.println("Area of r2: "+r2.area());
```

```
System.out.println("Area of r3: "+r3.area());
```

```
}
```

```
}
```

Object Oriented Programming with Java

The Ideal Structure of an Object: *Two Concentric Circles Representation*

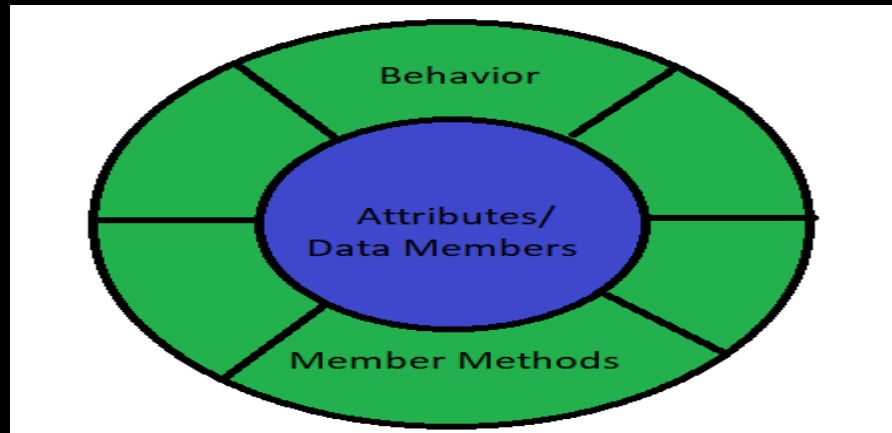
The ideal object structure can be visualized as composed of two concentric circles:

Inner Circle:

- private **data members** (**attributes** of the object).
- Data is hidden from direct access.
- Ensures data integrity and security.

Outer Circle:

- Exposes public **methods** (**behavior** of the object).
- Provides controlled access to the inner circle.
- Defines how the object interacts with the outside world.



Object Oriented Programming with Java

Remember the problem we encountered after making the attributes private:

Example:

```
class Rectangle {
    private int length;
    private int width;

    public int area() {
        return length * width;
    }
}
class RectangleMain {
    public static void main(String args[]) {
        Rectangle r1 = new Rectangle();

        // Private data members are not allowed to be accessed outside the class.
        r1.length = 2;
        r1.width = 3;

        // Public member method is allowed to be accessed outside the class.
        System.out.println("Area of r1: "+r1.area());
    }
}
```

How will these attributes get the values?

Object Oriented Programming with Java

Getter and Setter Methods

*“Getter and Setter methods are special member methods of a class that provide **controlled access** to an object's private data members.”*

Getters (Accessor Methods)

- Return the value of a private attribute.
- Provide read-only access to the data.
- Typically named *getAttributeName()*.

Example:

```
// Getter method for 'length' attribute
// of 'Rectangle' class
public int getLength()
{
    return length;
}
```

Object Oriented Programming with Java

Getter and Setter Methods

Setters (Mutator Methods)

- Set/ Modify the value of a private attribute.
- Allow validation before assignment.
- Typically named *setAttributeName()*.

Example:

```
// Setter method for 'length' attribute of  
// 'Rectangle' class  
public void setLength(int len)  
{  
    length = len;  
}
```

Object Oriented Programming with Java

CamelCase Notation

*“CamelCase is a **naming convention** in which multiple words may be written together without spaces/ underscores.”*

lowerCamelCase

- The first word starts with a lowercase letter, and subsequent words start with an uppercase letter.
- Used for: attribute names, method names.

Examples:

length, studentName, calculateTotalMarks()

UpperCamelCase

- Every word starts with an uppercase letter.
- Used for: class names, interface names.

Examples:

Rectangle, BankAccount

Object Oriented Programming with Java

Getter and Setter Methods

Example:

```
class Rectangle {  
    private int length;  
    private int width;  
  
    public void setLength(int len) {  
        length = len;  
    }  
  
    public int getLength() {  
        return length;  
    }  
  
    public void setWidth(int wid) {  
        width = wid;  
    }  
}
```

```
    public int getWidth() {  
        return width;  
    }  
  
    int area() {  
        return length * width;  
    }  
}
```

Object Oriented Programming with Java

Problem Solved !

Example:

```
class RectangleMain {  
    public static void main(String args[]) {  
        Rectangle r1 = new Rectangle();  
  
        r1.setLength(2);  
        r1.setWidth(3);  
  
        System.out.println("Length of r1: " + r1.getLength());  
        System.out.println("Width of r1: " + r1.getWidth());  
        System.out.println("Area of r1: " + r1.area());  
    }  
}
```

Setters are used
to assign
values.

Getters are
used to
access values.

Object Oriented Programming with Java

Getters & Setters vs. Direct Access – *Is There a Difference?*

At first, it may seem like getters and setters are just extra steps for accessing data, but they offer several advantages:

Controlled Access

- Direct access to private variables is not allowed.
- Getters and setters allow us to define rules for accessing and modifying data.

=> *Data Validation*: We can validate values before assigning them.

Example: Preventing negative values for length and width:

```
public void setLength(int len) {  
    if (len > 0) {  
        length = len;  
    } else {  
        System.out.println("Length must be positive");  
    }  
}
```

Object Oriented Programming with Java

Instance Variable Hiding Problem

*“When local variables (including formal parameters to methods) have the same name as instance variables, the **local variables hide the instance variables**.”*

This makes it impossible to directly access the instance variable from within the method.

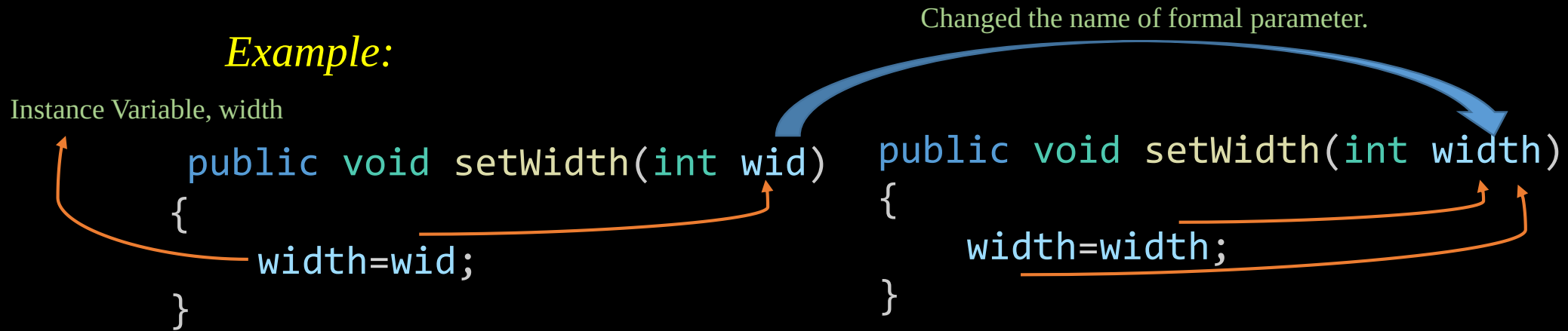
Example:

Instance Variable, width

```
public void setWidth(int wid)
{
    width=wid;
}
```

Changed the name of formal parameter.

```
public void setWidth(int width)
{
    width=width;
}
```



Object Oriented Programming with Java

'this' Keyword

“ *'this'* keyword is used in a method to *refer to the current object* i.e. *this is a reference to the object on which the method was invoked/ called.* ”

This makes it impossible to directly access the instance variable from within the method.

Example:

```
public void setWidth(int width)
{
    this.width = width;
}
```

Refers to the 'width' attribute of the current object.

*Instance Variable
Hiding Problem solved !*

Object Oriented Programming with Java

Polymorphism

“One thing, many forms.”

“Polymorphism gives the ability to an object/ method to take on many forms.”

=> Polymorphism is one of the core concepts of Object-Oriented Programming (OOP) .

Example:

A Person (One Thing) in Different Roles (Multiple Forms):

student, player, customer etc.

An add method (One Thing) can perform multiple functions (Multiple Forms):

adding two number, adding three numbers etc.

Object Oriented Programming with Java

Method Overloading

“Method Overloading allows multiple methods in the same class to have the same name but with different parameters (different type, number, or both).”

- It eliminates the need to create multiple method names for similar functionality.
- It improves code readability.
- It allows calling a method with different argument types.
- It is a way to achieve polymorphism in Java.

Rules of Method Overloading

- ✓ The method name must be the same.
- ✓ The method signature must be different (different number, type, or both).
- ✓ Return type does NOT differentiate overloaded methods.
- ✓ Overloaded methods must be in the same class.

Object Oriented Programming with Java

Method Overloading

Example:

```
class MathOperations {  
  
    // Method with one parameter  
    int add (int a) {  
        return a * a;  
    }  
  
    // Method with two parameters  
    int add (int a, int b) {  
        return a * b;  
    }  
  
    // Method with three parameters  
    int add (int a, int b, int c) {  
        return a * b * c;  
    }  
}
```

Object Oriented Programming with Java

Constructor

“A constructor in Java is a *special method* that is *called automatically* at the time of object creation to *initialize the object*. ”

Need of Constructors:

- They *automatically initialize objects* with default or specific values at the time of creation.
- They *eliminate the need for separate methods to assign initial values* after the object is created.
- They ensure that every object is created in a valid state, i.e.
The *data members of the object have proper initial values* immediately upon creation.

Object Oriented Programming with Java

Constructor

Properties of Constructors:

- The *name of a constructor* must be the *same as the name of the class* in which it is defined.
- A constructor *does not have a return type*, not even void.
- A constructor is *called automatically* when an object of the class is created.
- Constructors can be *parameterized or non-parameterized*, depending on whether they accept arguments or not.
- Java allows *constructor overloading*, which means a class can have multiple constructors with different parameter lists.

Note:

- ✓ When we do not explicitly define a constructor for a class, then *Java creates a default constructor* for the class.
- ✓ This default constructor *initializes the instance variables or data members with default values* which are zero, null, and false for numeric types, reference types and boolean.

Object Oriented Programming with Java

Constructor

Example:

```
class Rectangle {  
    private int length;  
    private int width;
```

```
    Rectangle() {  
        this.length = 1;  
        this.width = 1;  
    }
```

```
    int area() {  
        return length * width;  
    }  
}
```

// Object Creation Statement

```
Rectangle r1 = new Rectangle();
```

Constructor Calling



Object Oriented Programming with Java

Constructor

Constructor Overloading:

- As a constructor is a special type of method, it can be overloaded in the same way as other methods.
- **Constructor Overloading** means having *multiple constructors* defined within the same class.
- Each of these constructors must have a *different parameter list*, which may vary by the number, type, or order of parameters.
- The main purpose of constructor overloading is to *create objects in different ways*, depending on the situation or the initial values required during object creation.

Object Oriented Programming with Java

Constructor

Example of Constructor Overloading:

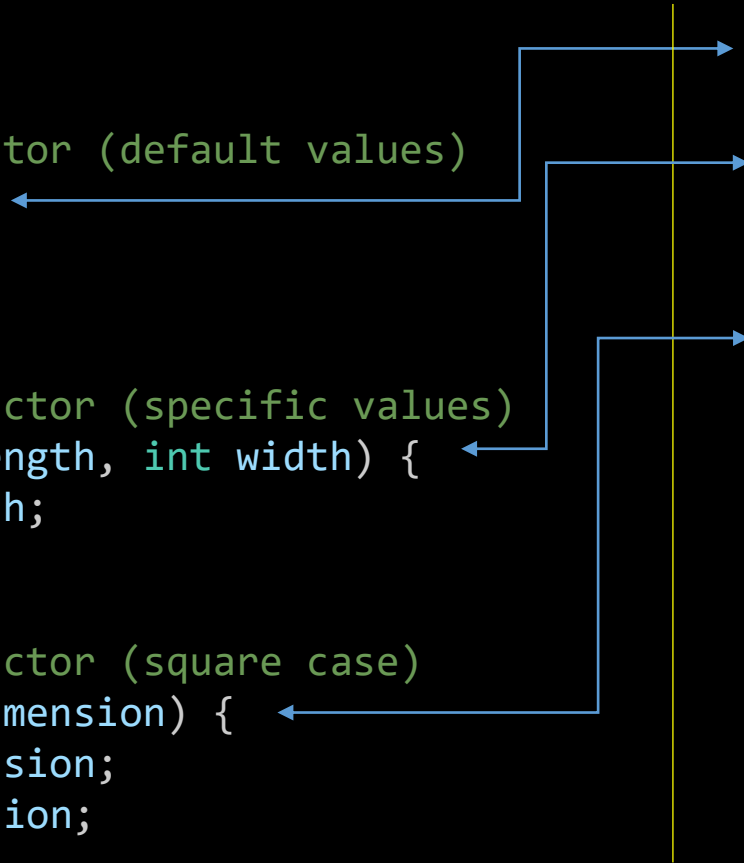
```
class Rectangle {  
    private int length;  
    private int width;  
  
    // No-argument constructor (default values)  
    public Rectangle() {  
        this.length = 1;  
        this.width = 1;  
    }  
  
    // Two-argument constructor (specific values)  
    public Rectangle(int length, int width) {  
        this.length = length;  
        this.width = width;  
    }  
  
    // One-argument constructor (square case)  
    public Rectangle(int dimension) {  
        this.length = dimension;  
        this.width = dimension;  
    }  
}
```

// Object Creation Statements

Rectangle r1 = new Rectangle();

Rectangle r2 = new Rectangle(4, 5);

Rectangle r3 = new Rectangle(6);



```

class Rectangle {
    private int length;
    private int width;

    Rectangle()
    {
        length=2;
        width=2;
    }
    Rectangle(int length,int width)
    {
        this.length = length;
        this.width = width;
    }
    Rectangle(int dim)
    {
        this.length = dim;
        this.width = dim;
    }

    public void setLength(int len)
    {
        if(len>0)
            length=len;
        else
        {
            System.out.println("Invalid value for length");
            length=1;
        }
    }

    public int getLength()
    {
        return length;
    }

    public void setWidth(int width)
    {
        this.width=width;
    }

    public int getWidth()
    {
        return width;
    }

    int area() {
        return length * width;
    }
}

class RectangleMain {
    public static void main(String args[]) {
        Rectangle r1 = new Rectangle();
        System.out.println("Length of r1: "+r1.getLength());
        System.out.println("Width of r1: "+r1.getWidth());
        System.out.println("Area of r1: " + r1.area());
    }
}

```

```
Rectangle r2 = new Rectangle(5,6);
System.out.println("Length of r2: "+r2.getLength());
System.out.println("Width of r2: "+r2.getWidth());
System.out.println("Area of r2: " + r2.area());

Rectangle r3 = new Rectangle(7);
System.out.println("Length of r3: "+r3.getLength());
System.out.println("Width of r3: "+r3.getWidth());
System.out.println("Area of r3: " + r3.area());

// Private data members are not allowed to be accessed
outside the class.
//r1.setLength(2);
//r1.setWidth(3);

//System.out.println("Length of r1: "+r1.getLength());
/*System.out.println("Width of r1: "+r1.getWidth());*/
//System.out.println("Area of r1: " + r1.area());
    }
}
```

Object Oriented Programming with Java

Static Members

“The 'static' keyword allows class members (data members and methods) to be used independently of objects. A static member belongs to the class, not to any specific object.”

Example: main() method is static because it must run before any object is created.

- *Access a method or variable without creating an object.*
- *Useful for utility methods or shared data.*
- *Reduces redundancy, as the same static member is shared across all instances.*

Object Oriented Programming with Java

Static Members

Static Data Members

- A static data member is declared using the *static keyword*.
- There is only *one copy of a static data member*, and it is shared among all objects of the class.
- It acts like a *global variable within the class*, accessible by all instances.

Example:

```
static int count;
```

Object Oriented Programming with Java

Static Members

Static Member Methods

- A static method is declared using the *static keyword*.
- A static method can be *called without creating an object* of the class.
- Static methods have some *restrictions*:
 - They can directly *access only other static members* (variables or methods) of the class.
 - They *cannot use the this or super keywords*, as static methods are not tied to any specific object instance.

Example:

```
static void show() {  
    System.out.println("Static Method");  
}
```

Object Oriented Programming with Java

Static Members

Static Blocks

- The *static block runs only once*, and that too when the class is loaded into memory for the first time, before any object is created.
- A static block is used to *initialize static variables* that require complex logic or computation during initialization.

Example:

```
static {  
    System.out.println("static block.");  
    b = a * 4;  
}
```

Object Oriented Programming with Java

Static Members

Accessing Static Members

The static members are accessed by using the class name followed by the dot operator and the member name.

Static members may also be accessed through objects, but it is better to access them using the class name to emphasize that they belong to the class, not to any specific object.

Example:

```
StaticCls.show();  
StaticCls.count;
```

Object Oriented Programming with Java

Static Members

Example:

```
class Rectangle {  
  
    // Static data member  
    static int objectCount;  
  
    // Static block (executed once when class is loaded)  
    static {  
        System.out.println("Rectangle class loaded.");  
        objectCount = 0;  
    }  
  
    // Static method  
    static void showObjectCount() {  
        System.out.println("Total Rectangle objects  
                           created: " + objectCount);  
    }  
}
```

```

class Rectangle {
    private int length;
    private int width;
    static int objectCount;

    static {
        objectCount=0;
        System.out.println("Static block executed");
    }

    static void showObjectCount()
    {
        System.out.println("Count of Rectangle Objects:
"+objectCount);
    }

    Rectangle()
    {
        length=2;
        width=2;
        objectCount++;
    }
    Rectangle(int length,int width)
    {
        this.length = length;
        this.width = width;
        objectCount++;
    }
    Rectangle(int dim)
    {
        this.length = dim;
        this.width = dim;
        objectCount++;
    }

    public void setLength(int len)
    {
        if(len>0)
            length=len;
        else
        {
            System.out.println("Invalid value for length");
            length=1;
        }
    }

    public int getLength()
    {
        return length;
    }

    public void setWidth(int width)
    {
        this.width=width;
    }

    public int getWidth()
    {

```

```

        return width;
    }

    int area() {
        return length * width;
    }
}

class RectangleMain {
    public static void main(String args[]) {
        Rectangle r1 = new Rectangle();
        System.out.println("Length of r1: "+r1.getLength());
        System.out.println("Width of r1: "+r1.getWidth());
        System.out.println("Area of r1: " + r1.area());
        Rectangle.showObjectCount();

        Rectangle r2 = new Rectangle(5,6);
        System.out.println("Length of r2: "+r2.getLength());
        System.out.println("Width of r2: "+r2.getWidth());
        System.out.println("Area of r2: " + r2.area());
        Rectangle.showObjectCount();

        Rectangle r3 = new Rectangle(7);
        System.out.println("Length of r3: "+r3.getLength());
        System.out.println("Width of r3: "+r3.getWidth());
        System.out.println("Area of r3: " + r3.area());
        Rectangle.showObjectCount();

        // Private data members are not allowed to be accessed
        outside the class.
        //r1.setLength(2);
        //r1.setWidth(3);

        //System.out.println("Length of r1: "+r1.getLength());
        /*System.out.println("Width of r1: "+r1.getWidth());*/
        //System.out.println("Area of r1: " + r1.area());
    }
}

```

Object Oriented Programming with Java

Inheritance

*“Inheritance is a mechanism in Java by which **one class acquires the properties and behavior (data members and methods) of another class.**”*

Superclass: A class whose properties and behavior are inherited by another class.

Subclass: A class that inherits properties and behavior from another class.

*“A **subclass is a specialized version of a super class** . It inherits all of the members defined by the superclass and adds its own, unique elements.*

Example: A ‘Square’ class can inherit from a ‘Rectangle’ class since a square is a special type of rectangle with equal length and width.

Rectangle – Superclass.

Square – Subclass.

Object Oriented Programming with Java

Inheritance

“Inheritance Promotes Reusability — But Use It Wisely!”

Advantage: Inheritance allows us to reuse existing code.

Common properties and behaviors defined in a superclass can be reused by all its subclasses.

If both Teacher and Student have common attributes like name, address, and email, can we inherit student from teacher or vice versa?

Caution: Don't use inheritance just for code reuse. Always apply the "is-a" test.

Ask:

Teacher is a Student? ❌

Student is a Teacher? ❌

So, Inheritance is not justified here.

Object Oriented Programming with Java

Inheritance

Implementation in Java

To inherit a class, we incorporate the definition of one class into another by using the '*extends*' keyword.

```
class Super {  
    // data members and methods  
}  
  
class Sub extends Super {  
    // additional data members and methods  
}
```

Object Oriented Programming with Java

Inheritance

Types

Single Inheritance:

A class inherits from only one superclass.

Multilevel Inheritance:

A class inherits from a superclass, and that superclass itself inherits from another superclass.

Hierarchical Inheritance:

Multiple sub classes inherit from a single superclass.

Multiple Inheritance:

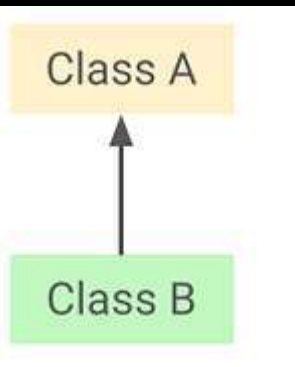
A class inherits from multiple super classes.

Java does not support multiple inheritance.

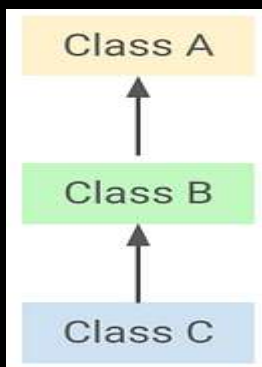
Hybrid Inheritance:

A combination of two or more types of inheritance.

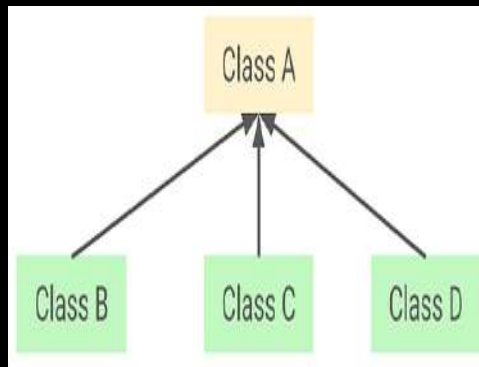
Java does not support hybrid inheritance if it involves multiple inheritance.



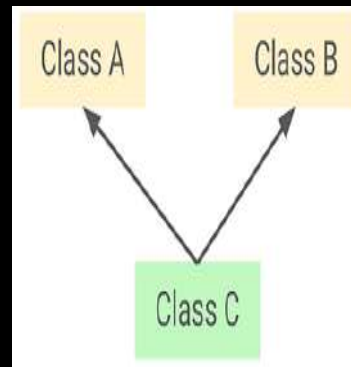
Single Inheritance



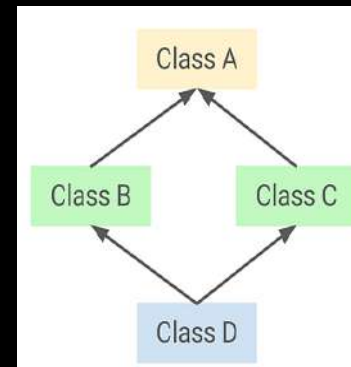
Multilevel Inheritance



Hierarchical Inheritance



Multiple Inheritance



Hybrid Inheritance

```
class Super
{
    int sup;

    void setSup(int sup)
    {
        this.sup = sup;
    }
    int getSup()
    {
        return sup;
    }
}
class Sub extends Super
{
    int sub;

    void setSub(int sub)
    {
        this.sub = sub;
    }
    int getSub()
    {
        return sub;
    }
}
class InheritanceDemo
{
    public static void main(String args[])
    {
        Sub obj = new Sub();
        obj.setSup(98);
        System.out.println(obj.getSup());

        obj.setSub(68);
        System.out.println(obj.getSub());
    }
}
```

Object Oriented Programming with Java

Inheritance

Accessing Private Members of Superclass

- ✓ Private members are accessible only within the class in which they are defined so
- ✓ ***Although a subclass includes all of the members of its superclass, it cannot directly access private data or methods of the superclass.***
- ✓ Subclasses can access private members of superclass using public/protected methods of the superclass (eg. getters/setters).

Example

```
class A {  
    private int x = 10;  
    public int getX() {  
        return x;  
    }  
}
```

```
class B extends A {  
    void display() {  
        // System.out.println(x);  
        // Error: x has private access in A  
        System.out.println(getX());  
        // Correct way to access  
    }  
}
```

Object Oriented Programming with Java

Inheritance

'protected' Access Specifier

- ✓ The *protected* access specifier allows a class member to be
 - accessed within the same class.
 - accessed by subclasses (child classes).
- ✓ It provides more access than private but less than public.
- ✓ Commonly used when we want to allow child classes to reuse or modify superclass members.

Example

```
class A {  
    protected int x = 10;  
    public int getX() {  
        return x;  
    }  
}
```

```
class B extends A {  
    void display() {  
        System.out.println(x);  
    }  
}
```

Object Oriented Programming with Java

Inheritance

Constructors in Inheritance

- ✓ The constructor of the superclass is always called when an object of the subclass is created.
- ✓ The superclass constructor initializes the inherited part of the object.
- ✓ Constructors are executed in the order of inheritance — from superclass to subclass. Java automatically inserts a call to the default constructor of the superclass.

Example

```
class A {  
    int x;  
    A() {  
        System.out.println(  
            "Super constructor.");  
        x = 10;  
    }  
}
```

```
class B extends A {  
    int y;  
    B() {  
        System.out.println(  
            "Sub constructor.");  
        x = 10;        y = 20;  
    }  
    void display() {  
        System.out.println(  
            x + " and " + y);  
    }  
}
```

Object Oriented Programming with Java

Inheritance

'super' keyword

Whenever a subclass needs to refer to its immediate superclass, it can use the super keyword.

'super' has two general uses in Java:

1. Calling Superclass Constructor

- ✓ Used to call the constructor of the superclass from within the subclass constructor.
- ✓ Must be the first statement in the subclass constructor.

```
super();           // calls the default constructor of superclass  
super(args);       // calls a parameterized constructor
```

2. Accessing Superclass Members

- ✓ Used to access a superclass method or variable that is hidden by a subclass member.

```
super.dataMember;  
super.memberMethod();
```

Object Oriented Programming with Java

Inheritance

'super' keyword

Example

```
class A {  
    int x;  
    A() {  
        x = 10;  
    }  
    void display() {  
        System.out.println(x);  
    }  
}
```

```
class B extends A {  
    int y;  
    B() {  
        // Calling Superclass constructor  
        super();  
        y = 20;  
    }  
  
    void display() {  
        // Calling Super class member method  
        super.display();  
        System.out.println(y);  
    }  
}
```

```

class A
{
    //private int x=10;
    protected int x;

    A()
    {
        x=0;
    }

    A(int x)
    {
        System.out.println("Super Class Constructor");
        this.x=x;
    }

    int getX()
    {
        return x;
    }
    void display()
    {
        System.out.println(x);
    }
}

class B extends A
{
    int y;
    B(int x, int y)
    {
        super(x);
        System.out.println("Sub class Constructor");
        this.y=y;
    }
    //void display()
    //{
        //System.out.println(x + " and " + y);
        //System.out.println(getX());
    //}
    void display()
    {
        super.display();
        System.out.println(y);
    }
}

class AccessDemo
{
    public static void main(String args[])
    {
        B obj = new B(5,6);
        obj.display();
    }
}

```

Object Oriented Programming with Java

Inheritance

Method Overriding

Method Overriding occurs when a subclass defines a method that already exists in its superclass with the same signature (name, parameters, and return type).

- ✓ When an overridden method is called from the subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.
- ✓ It allows a subclass to provide its own version of a method and enables runtime polymorphism

Rules for Method Overriding

1. The method must have the **same name, return type, and parameters** as the superclass method.
2. The method **must be inherited** from the superclass.
3. The overridden method **cannot be more restrictive** i.e. it must allow at least the same or more access than the method in the superclass.
4. We use the **@Override annotation** (optional but recommended) to indicate overriding.

Object Oriented Programming with Java

Inheritance

Method Overriding

Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class OverridingDemo {  
    public static void main(String[] args) {  
        Dog dog = new Dog();  
        dog.sound(); // Output: Dog barks  
    }  
}
```

Object Oriented Programming with Java

Inheritance

Dynamic Method Dispatch

“Which **method** to **dispatch** the call to is determined **dynamically** at runtime.”

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.

It is a way to achieve *run-time polymorphism*.

“A superclass reference variable can refer to a subclass object.”

- Java uses this fact to resolve calls to overridden methods at run time.
- When an overridden method is called through a superclass reference, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs. This determination is made at run time.
- When different types of objects are referred to, different versions of an overridden method will be called achieving polymorphism.

“It is the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.”

Object Oriented Programming with Java

Inheritance

Dynamic Method Dispatch - Example

```
class Shape {  
    void draw() {  
        System.out.println("Drawing a shape");  
    }  
}
```

```
class Rectangle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a  
                           rectangle");  
    }  
}
```

```
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a  
                           circle");  
    }  
}
```

```
public class DMDMain {  
    public static void main(String[] args) {  
        Shape shape = new Shape();  
        Rectangle rectangle = new Rectangle();  
        Circle circle = new Circle();  
  
        Shape shapeRef;  
  
        shapeRef = shape;  
        shapeRef.draw(); // Output: Drawing a shape  
        shapeRef = rectangle;  
        shapeRef.draw(); // Output: Drawing a rectangle  
        shapeRef = circle;  
        shapeRef.draw(); // Output: Drawing a circle  
    }  
}
```

Object Oriented Programming with Java

Inheritance

AKTU PYQ

Illustrate polymorphism and its types in Java. Differentiate between run-time and compile-time polymorphism. Write super class Shape with method displayArea() and sub class Rectangle. Demonstrate method overriding with this example.

Compile-Time Polymorphism (Static Binding):

Achieved using method overloading. Method call is resolved at compile time.

Run-Time Polymorphism (Dynamic Binding):

Achieved using method overriding. Method call is resolved at run time using dynamic method dispatch.

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
<i>Mechanism</i>	Method Overloading	Method Overriding
<i>Method Resolution Time</i>	At Compile Time	At Run Time
<i>Flexibility</i>	Less flexible	More flexible (based on object type)
<i>Involvement of Inheritance</i>	Not necessary	Requires Inheritance

Object Oriented Programming with Java

Inheritance

AKTU PYQ

Write super class Shape with method displayArea() and sub class Rectangle. Demonstrate method overriding with this example.

```
class Shape {  
    void displayArea() {  
        System.out.println("Area of shape is undefined");  
    }  
}  
  
class Rectangle extends Shape {  
    int length;  
    int width;  
  
    Rectangle(int length, int width) {  
        this.length = length;  
        this.width = width;  
    }  
}
```

Object Oriented Programming with Java

Inheritance

AKTU PYQ

Write super class Shape with method displayArea() and sub class Rectangle. Demonstrate method overriding with this example.

```
@Override
void displayArea() {
    int area = length * width;
    System.out.println("Area of Rectangle: " + area);
}

public class ShapeMain {
    public static void main(String[] args) {
        Shape shape;
        Rectangle rectangle = new Rectangle(3,4);
        shape = rectangle;
        shape.displayArea(); // Calls overridden method
    }
}
```

```
class Animal
{
    void sound()
    {
        System.out.println("Animal makes sound");
    }
}
class Cat extends Animal
{
    @Override
    void sound()
    {
        System.out.println("Cat meows");
    }
}
class AnimalMain
{
    public static void main(String args[])
    {
        Cat cat = new Cat();
        cat.sound();
    }
}
```

```
class Shape
{
    void draw()
    {
        System.out.println("Drawing a shape");
    }
}
class Rectangle extends Shape
{
    void draw()
    {
        System.out.println("Drawing a rectangle");
    }
}
class Square extends Shape
{
    void draw()
    {
        System.out.println("Drawing a square");
    }
}
class ShapeMain
{
    public static void main(String args[])
    {
        Shape shape = new Shape();
        Rectangle rectangle = new Rectangle();
        Square square = new Square();

        Shape ref;

        ref = shape;
        ref.draw();

        ref = rectangle;
        ref.draw();

        ref = square;
        ref.draw();
    }
}
```

```

class Shape
{
    void displayArea()
    {
        System.out.println("Area undefined");
    }
}
class Rectangle extends Shape
{
    int length;
    int width;

    Rectangle(int length, int width)
    {
        this.length = length;
        this.width = width;
    }
    @Override
    void displayArea()
    {
        int area = length * width;
        System.out.println("Area of Rectangle: "+area);
    }
}
class PYQShapeMain
{
    public static void main(String args[])
    {
        Shape shape;

        Rectangle rectangle = new Rectangle(5,6);

        shape = rectangle;

        shape.displayArea();

        Shape shapeObj = new Shape();
        shape = shapeObj;

        shape.displayArea();

    }
}

```

Object Oriented Programming with Java

Inheritance

Abstraction

*Abstraction is the process of hiding unnecessary details and showing only essential information. Abstraction can be achieved with either **abstract classes** or **interfaces**.*

Abstract Method

*An abstract method has no body (no definition/ implementation). It is declared using the keyword **abstract**, and ends with a semicolon.*

Example:

```
abstract void display();
```

Abstract Class

*An abstract class is a class that can not be instantiated i.e. we can not create its objects. It needs to be inherited by another class to use its properties and methods. An abstract class is declared using the “**abstract**” keyword in its class definition.*

Example:

```
abstract class Shape {  
    . . .  
}
```

Object Oriented Programming with Java

Inheritance

Abstract Class

- ✓ A class containing abstract method must be declared abstract.
- ✓ Abstract method must be overridden in the subclass, else the subclass must also be declared abstract.
- ✓ Abstract classes may contain both:
 - Abstract methods (without definition/ implementation)
 - Concrete methods (with definition/ implementation)
- ✓ Abstract class defines a generalized form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details.
- ✓ We can not create objects of an abstract class because such objects would be useless, because an abstract class is not fully defined.

Object Oriented Programming with Java

Inheritance

Abstract Class - Example

```
abstract class Shape {  
    abstract void displayArea();  
}
```

```
class Rectangle extends Shape {  
    int length;  
    int width;  
  
    Rectangle(int length, int width) {  
        this.length = length;  
        this.width = width;  
    }  
    @Override  
    void displayArea() {  
        int area = length * width;  
        System.out.println("Area of Rectangle: " + area);  
    }  
}
```

Object Oriented Programming with Java

Inheritance

AKTU PYQ

```
public class ShapeMain {  
    public static void main(String[] args) {  
        Shape shape;  
        Rectangle rectangle = new Rectangle(3,4);  
        shape = rectangle;  
        shape.displayArea(); // Calls overridden method  
    }  
}
```

```
abstract class Shape
{
    abstract void displayArea();
}
class Rectangle extends Shape
{
    int length;
    int width;

    Rectangle(int length, int width)
    {
        this.length = length;
        this.width = width;
    }
    @Override
    void displayArea()
    {
        int area = length * width;
        System.out.println("Area of Rectangle: "+area);
    }
}
class AbstractShapeMain
{
    public static void main(String args[])
    {
        Shape shape;

        Rectangle rectangle = new Rectangle(5,6);

        shape = rectangle;

        shape.displayArea();
    }
}
```

Object Oriented Programming with Java

‘final’ keyword

The keyword *final* in Java is used to *restrict modification*.

- It can be **applied to variables, methods, and classes**.
- Each usage of *final* serves a different purpose in preventing changes.

‘final’ variables

A variable declared as *final* **cannot be reassigned** after it has been initialized.

- It acts as a **constant** value.
- *final* variables must be **initialized at the time of declaration or in a constructor** (if it is a data member/ attribute/ instance variable).
- The **Method parameters** can also be declared *final*. Declaring a parameter *final* prevents it from being changed within the method.
- It is a common coding convention to write the name of *final* variable in all **uppercase** letters.

Example:

```
final int MIN_VALUE = 1;  
// ✗ Error: Cannot assign a value to final variable  
MIN_VALUE = 2;
```

Object Oriented Programming with Java

‘final’ keyword

‘final’ methods

A final method *cannot be overridden* by any subclass.

- It is used when you want to prevent a subclass from modifying the behavior of a method.

Example:

```
class SuperCls {  
    final void show() {  
        System.out.println("This is a final method.");  
    }  
}  
  
class SubCls extends SuperCls {  
    // ✗ Error: Cannot override the final method  
    void show() {  
        System.out.println("Trying to override.");  
    }  
}
```

Object Oriented Programming with Java

‘final’ keyword

‘final’ classes

A class declared as final *cannot be extended or inherited* (i.e., it cannot have subclasses).

- It is used when you want to prevent inheritance for security or design reasons.

Example:

```
final class SuperCls {  
    void show() {  
        System.out.println("This is a final method.");  
    }  
}  
// ✗ Error: Cannot extend or inherit the final class  
class SubCls extends SuperCls {  
  
}
```

```

final class MyClass
{
    final int attr;

    MyClass(int attr)
    {
        this.attr =attr;
    }

    final void myMethod(final int param)
    {
        final int localVar=3;

        //localVar=6;
        //attr=7;
        //param=8;

        System.out.println("localVar = "+localVar);
        System.out.println("param = "+param);
        System.out.println("attr = "+attr);

    }
}

class MySubClass extends MyClass
{
    MySubClass(int attr)
    {
        super(attr);
    }
    void myMethod(int param)
    {
        //System.out.println(localVar);
        System.out.println(param);
        System.out.println(attr);
    }
}

class FinalMain
{
    public static void main(String args[])
    {
        //MyClass obj = new MyClass(2);
        //obj.myMethod(4);
        MySubClass subObj = new MySubClass(4);
        subObj.myMethod(38);
    }
}

```

Object Oriented Programming with Java

Interface

An interface in Java is a collection of *abstract methods*.

- Along with abstract methods, interface may contain *constants* and *default, static, & private methods*.
- It acts as a *blueprint for classes* that wish to implement it.
- Interfaces *cannot be instantiated* directly i.e. we cannot create its objects.

Defining an Interface

An interface is defined using the *interface* keyword. The method inside the interface is implicitly *public and abstract* (unless declared default, static or private).

The interface itself may be declared with public or no access specifier (default).

Example:

```
interface IOperations {  
    int add(int a, int b);  
    int multiply(int a, int b);  
}
```

Object Oriented Programming with Java

Interface

Implementing an Interface

A *class* implements an interface using the *implements* keyword.

- It must provide *definitions for all abstract methods* of the interface.
- If a class implements more than one interface, the interfaces are *separated with a comma*.
- The methods that implement an interface must be declared *public*.
- The *type signature* of the implementing method must match exactly the type signature specified in the interface definition.
- If a class implements an interface but does not define all the methods required by that interface, then that class must be declared as *abstract*.

Object Oriented Programming with Java

Interface

Implementing an Interface

Example:

```
class ClsOperations implements IOperations {  
  
    public int add(int a, int b) {  
        return a+b;  
    }  
  
    public int multiply(int a, int b) {  
        return a*b;  
    }  
  
    //Class' own method other than defined in the interface  
    int subtract(int a, int b) {  
        return a-b;  
    }  
  
}
```

Object Oriented Programming with Java

Interface

Need of Interfaces

- Using interface, we can specify what a class must do, but not how it does it.
- Interfaces support the OOP concept of *Abstraction*. By using interfaces, a class can hide internal implementation and expose only the required functionalities.
- Once an interface is defined, any number of classes can implement an interface. Also, one class can implement any number of interfaces.
- To implement an interface, a class must provide the complete set of methods required by the interface. However, each class is free to determine the details of its own implementation.

Object Oriented Programming with Java

Interface

Variables in Interfaces

Variables can be declared inside interface declarations.

- They are implicitly public, static, and final meaning they cannot be changed by the implementing class.
- They must also be initialized.
- When a class implements the interface, all of the variable names will be in scope of the class as final variables (constants).

Example:

```
interface IOperations {  
    int SCALE_FACTOR = 3;  
  
    int add(int a, int b);  
    int multiply(int a, int b);  
    int scale(int a);  
}
```

Object Oriented Programming with Java

Interface

Interface References

*We can not create objects of interface but we can declare **interface references**. Any instance of any class that implements the declared interface can be referred to by such a reference.*

- When a method is called through interface references, the correct version will be called based on the **actual instance of the class being referred** to.
- An interface reference variable has **knowledge only of the methods declared by its interface** declaration so through this reference, we can not call other members of the class that do not belong to interface.

Object Oriented Programming with Java

Interface

Interface References

Example:

```
interface IOperations {
    int add(int a, int b);
}
class ClsOperations1 implements IOperations {
    public int add(int a, int b) {
        return a + b;
    }
}
class ClsOperations2 implements IOperations {
    public int add(int a, int b) {
        return a*a + b*b;
    }
}
```

```
public class InterfaceDemo {
    public static void main(String[] args) {
        IOperations ref;
        ClsOperations1 obj1 = new ClsOperations1();
        ClsOperations2 obj2 = new ClsOperations2();

        ref=obj1;
        System.out.println(ref.add(2,3));
        ref=obj2;
        System.out.println(ref.add(2,3));
    }
}
```

```

interface IOperations
{
    int SCALE_FACTOR = 2;

    //int scale(int a);
    //int add(int a,int b);
    int multiply(int a, int b);
}
class ClsOperations implements IOperations
{
    public int add(int a,int b)
    {
        return a+b;
    }
    public int scale(int a)
    {
        //SCALE_FACTOR=3;
        return a*SCALE_FACTOR;
    }
    int subtract(int a, int b)
    {
        return a-b;
    }
    public int multiply(int a, int b)
    {
        return a*b;
    }
}
class MyClass implements IOperations
{
    public int multiply(int a, int b)
    {
        return (a*a)*(b*b);
    }
}
class InterfaceMain
{
    public static void main(String args[])
    {
        IOperations ref;
        MyClass obj1 = new MyClass();
        ClsOperations obj2 = new ClsOperations();

        ref=obj2;
        System.out.println(ref.multiply(3,4));
        System.out.println(ref.add(3,4));

        ref=obj1;
        System.out.println(ref.multiply(3,4));

        //System.out.println(obj.multiply(3,4));
        //System.out.println(obj.subtract(3,4));
        //System.out.println(obj.scale(3));
    }
}

```

Object Oriented Programming with Java

Interface

Interface Inheritance

An interface can inherit another interface using the extends keyword.

An interface can extend one or more interfaces (i.e., multiple interface inheritance is allowed).

Example:

```
interface A {  
    void methodA();  
}  
interface B {  
    void methodB();  
}  
interface C extends A, B {  
    void methodC();  
}
```

Object Oriented Programming with Java

Interface

Interface Inheritance

If a class implements a sub-interface, it must implement all the methods of:

- the sub-interface itself, and
- all of its super interfaces.

Example:

```
class MyClass implements C {  
    public void methodA() { ... }  
    public void methodB() { ... }  
    public void methodC() { ... }  
}
```

Object Oriented Programming with Java

Interface

Multiple Interface Implementation

Java does not support multiple inheritance with classes but supports implementing multiple interfaces.

Example:

```
interface A {  
    void show();  
}  
  
interface B {  
    void show();  
}  
  
class MyClass implements A, B {  
    public void show() {  
        System.out.println("Resolved show method");  
    }  
}
```

Object Oriented Programming with Java

Interface

Default Methods

A default method (added in JDK 8) define a *default implementation* for an interface method. By use of a default method, it is possible for an interface method to provide a body, rather than being abstract. A default method is defined using *default keyword*.

- Default methods allow interfaces to *evolve* without breaking existing implementations.
- Default method may be used to specify *optional methods* in an interface.

Example:

```
interface IOperations {  
    int SCALE_FACTOR = 3;  
  
    int add(int a, int b);  
    int multiply(int a, int b);  
  
    default int scale(int a) {  
        return a * SCALE_FACTOR;  
    }  
}
```

Object Oriented Programming with Java

Interface

Static Methods

Like static methods in a class, a static method defined by an interface (added in JDK 8) can be called independently of any object.

- *No implementation* of the interface is necessary, and *no instance* of the interface is required, in order to call a static method.
- Instead, a static method is *called by the interface name*, followed by a period, followed by the method name.

Example:

```
interface Utility {  
    static void greet() {  
        System.out.println("Hello!");  
    }  
}  
  
public class InterfaceDemo {  
    public static void main(String[] args) {  
        Utility.greet();  
    }  
}
```

Object Oriented Programming with Java

Interface

Private Methods

A private interface method (added in JDK 9) *can be called by a default method or another private method* defined by the same interface.

- Private interface method *cannot be used by code outside the interface* in which it is defined.
- Private method lets two or more default methods use a common piece of code, *avoiding code duplication*.

Example:

```
interface IDBOps {  
    default void insert() {  
        System.out.println("Record Inserted !");  
        log("INSERT");  
    }  
    default void update() {  
        System.out.println("Record Udpated !");  
        log("UPDATE");  
    }  
    private void log(String op) {  
        System.out.println("Operation Logged: "+op);  
    }  
}
```

```
interface A
{
    void methodA();
}
interface B
{
    void methodB();
}
interface C extends A,B
{
    void methodC();
}
class MyClass implements C
{
    public void methodC()
    {
        System.out.println("Method C's Definition");
    }
    public void methodB()
    {
        System.out.println("Method B's Definition");
    }
    public void methodA()
    {
        System.out.println("Method A's Definition");
    }
}
class InterfaceConceptsMain
{
    public static void main(String args[])
    {
        MyClass obj = new MyClass();
        obj.methodC();
        obj.methodB();
        obj.methodA();
    }
}
```

```
interface A
{
    void show();
    void methodA();
}
interface B
{
    void show();
    void methodB();
}
class MyClass implements A,B
{
    public void show()
    {
        System.out.println("The show() method");
    }
    public void methodA()
    {
        System.out.println("The methodA() method");
    }
    public void methodB()
    {
        System.out.println("The methodB() method");
    }
}
class MultipleInterfaceImplementation
{
    public static void main(String args[])
    {
        MyClass obj = new MyClass();
        obj.show();
        obj.methodA();
        obj.methodB();
    }
}
```

```
interface IOperations
{
    int add(int a, int b);

    default int scale(int a)
    {
        return a*2;
    }

    static int subtract(int a,int b)
    {
        return a-b;
    }
}
class ClsOperations implements IOperations
{
    public int add(int a,int b)
    {
        return a+b;
    }
    public int scale(int a)
    {
        return a*3;
    }
}
class DefaultMain
{
    public static void main(String args[])
    {
        ClsOperations obj = new ClsOperations();
        System.out.println(obj.add(6,6));
        System.out.println(obj.scale(3));
        System.out.println(IOperations.subtract(3,1));
    }
}
```

```
interface IDBOps
{
    default void insert()
    {
        System.out.println("Insert method called");
        log("INSERT");
    }
    default void delete()
    {
        System.out.println("Delete method called");
        log("DELETE");
    }
    private void log(String operation)
    {
        System.out.println("The Operation logged is "+operation);
    }
}
class MyClass implements IDBOps
{
}

class PrivateMain
{
    public static void main(String args[])
    {
        MyClass obj = new MyClass();
        obj.insert();
        obj.delete();
        //obj.log("Main");
    }
}
```

Object Oriented Programming with Java

Arrays

*“An array in Java is a **collection of elements** of the **same data type**.”*

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- The elements in an array are indexed, starting from 0 to (length - 1).

Defining an Array

Defining an array in Java is a two-step process:

1. You must **declare a variable** of the desired array type.
2. You must **allocate the memory** that will hold the array, using **new**, and assign it to the array variable.
In Java, all arrays are dynamically allocated.

Example:

```
int numbers[] = new int[5]; OR  
int[] numbers = new int[5];
```

The array may be initialized with values at the time of declaration as follows:

```
int[] numbers = {90, 85, 75, 88, 92};
```

Object Oriented Programming with Java

Arrays

Accessing elements of an Array

Each element in the array is accessed using its *index*.

Index starts from 0

so, numbers[0] gives the first element and numbers[4] gives the last one (if length is 5).

Example:

```
System.out.println(numbers[3]); // prints 88, the fourth element
```

Modifying an element:

```
numbers[2] = 80; // changes third element to 80
```

Note: Accessing an element from the index outside the range of the array leads to an error in Java.

```
System.out.println(numbers[5]);  
//Runtime Error: ArrayIndexOutOfBoundsException
```

Always make sure to access indexes from 0 to array.length - 1 to avoid this runtime exception.

Object Oriented Programming with Java

Arrays

Arrays as Objects in Java

In Java, arrays are treated as objects, regardless of the data type of the elements they store. When we declare an array using `new`, memory is dynamically allocated at runtime.

The length Property of Array Object

Every array object in Java has a built-in property called *length*, which returns the number of elements in the array.

Example:

```
System.out.println(numbers.length); // prints 5
```

Note: `length` is a property of Array, not a method — so we do not use parentheses after it.

Object Oriented Programming with Java

Arrays

Multidimensional Arrays

The array in Java may have more than one dimension. The multidimensional arrays are implemented as arrays of arrays.

To declare a multidimensional array, we specify each additional dimension using another set of square brackets.

Example:

```
int numbers[][] = new int[4][3]; //This allocates a 4 by 3 2d array
```

The multidimensional array may also be initialized with values at the time of declaration as follows:

```
int[][] numbers = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

Object Oriented Programming with Java

Arrays

Multidimensional Arrays

Internally, the 2d array is implemented as an array of arrays of type int.
So, the array declared in previous example is essentially an array of four 1d arrays.

```
numbers
|      0 1 2
0 |--> [1, 2, 3]
|
1 |--> [4, 5, 6]
|
2 |--> [7, 8, 9]
|
3 |--> [10, 11, 12]
```

The element from a 2d array may be accessed as follows:

```
System.out.println(numbers[1][2]); // Prints 6
```

```

class ArrayMain
{
    public static void main(String args[])
    {
        //int arr[]={1,2,3,4,5};
        int arr[] = new int[5];

        for(int i=0;i<arr.length;i++)
        {
            arr[i] = i+1;
        }

        int sum=0;

        //for each loop
        for(int num:arr)
        {
            sum = sum + num;
        }

        /*for(int i=0;i<=arr.length-1;i++)
        {
            sum = sum + arr[i];
        }*/
        System.out.println("Sum = "+sum);

        //int[][] my2dArray = new int[2][3];
        int[][] my2dArray = {{2,2,2},{2,2,2}};
        for(int i=0;i<2;i++)
        {
            for(int j=0;j<3;j++)
            {
                my2dArray[i][j] = 2;
            }
        }

        for(int i=0;i<2;i++)
        {
            for(int j=0;j<3;j++)
            {
                System.out.print(my2dArray[i][j]);
            }
            System.out.println("");
        }
    }
}

```

Object Oriented Programming with Java

String

“A String is a *sequence of characters*. ”

- In Java, **String** is a *built-in class*.
- A string is enclosed in *double quotes*, like “Hello”.

Defining a ‘String’ object

In Java, string literals (string values like "Hello") are automatically treated as objects of the String class. We can assign a string literal directly to a String reference.

Example:

```
String str = "Hello";
```

Java also allows creating String objects using *constructors* of the String class.

Examples:

```
String str1 = new String();           // Empty string object
String str2 = new String("Hello");    // From string literal

char[] chars = {'J', 'a', 'v', 'a'};
String str3 = new String(chars);      // From character array

byte[] bytes = {65, 66, 67};
String str4 = new String(bytes);      // From byte array (ASCII values)

String str5 = new String(str2);       // From another String object
```

Object Oriented Programming with Java

String

Features of String in Java

1. Strings are *objects* – created using the String class.
2. Strings are *immutable*, meaning once a String object is created, its content cannot be changed.

But then how were we able to “change” a String in the previous programs?

What happens is that when we modify a String using operations like concatenation (+), Java does not change the existing object. Instead, it creates a new String object with the modified content.

Example:

```
String s = "Hello";  
s = s + " World";    /* A new String object is created.  
Old "Hello" still exists. 's' now  
refers to "Hello World" - a new object  
in memory.*/
```

Object Oriented Programming with Java

String

Methods of String class

length()

Returns the length i.e. number of characters in the string.

Example:

```
String str = "Java";  
System.out.println(str.length()); // Output: 4
```

charAt(int index)

Returns the character at the specified index (0-based).

Example:

```
String str = "Hello";  
System.out.println(str.charAt(1)); // Output: e
```

Object Oriented Programming with Java

String

Methods of String class

substring(int beginIndex, int endIndex)

Returns a new string from beginIndex to endIndex - 1.

Example:

```
String str = "Programming";  
System.out.println(str.substring(0, 4)); // Output: Prog
```

toUpperCase() / toLowerCase()

Converts all characters to upper or lower case.

Example:

```
String str = "Java";  
System.out.println(str.toUpperCase()); // Output: JAVA  
System.out.println(str.toLowerCase()); // Output: java
```

Object Oriented Programming with Java

String

Methods of String class

equals(String another) / equalsIgnoreCase(String another)

Checks whether two strings are equal.

Example:

```
String s1 = "Hello";  
String s2 = "hello";  
System.out.println(s1.equals(s2));           // Output: false  
System.out.println(s1.equalsIgnoreCase(s2)); // Output: true
```

'equals()' vs. '=='

The equals() method and the == operator perform two different operations.

The equals() method compares the characters inside a String object.

The == operator compares two object references to see whether they refer to the same instance.

Example:

```
String s3 = "Java";  
String s4 = new String(s3);  
System.out.println(s3.equals(s4));           // Output: true  
System.out.println(s3==s4);                  // Output: false
```

Object Oriented Programming with Java

String

Methods of String class

indexOf(String sub) / lastIndexOf(String sub)

Finds the position of first or last occurrence of a substring.

Example:

```
String str = "banana";  
System.out.println(str.indexOf("an"));      // Output: 1  
System.out.println(str.lastIndexOf("an"));  // Output: 3
```

startsWith(String prefix) / endsWith(String suffix)

Checks if the string begins or ends with a specified substring.

Example:

```
String str = "HelloWorld";  
System.out.println(str.startsWith("Hello")); // Output: true  
System.out.println(str.endsWith("World"));   // Output: true
```

Object Oriented Programming with Java

String

Methods of String class

trim()

Removes any leading and trailing white spaces.

Example:

```
String str = "  Java  ";  
System.out.println(str.length());           // Output: 8  
System.out.println(str.trim());             // Output: "Java"  
System.out.println(str.trim().length());    // Output: 4
```

replace(char oldChar, char newChar)

Replaces all occurrences of one character with another.

Example:

```
String str = "apple";  
System.out.println(str.replace('p', 'b'));  
// Output: abble
```

```

class StringDemo
{
    public static void main(String args[])
    {

        String str = "Meerut";
        String str1 = new String();
        str1 = str;
        System.out.println(str1);
        String str2 = new String("Delhi");
        System.out.println(str2);

        char[] arr = {'P','u','n','e'};
        String str3 = new String(arr);
        System.out.println(str3);

        byte[] byteArr = {65,91,66,92};
        String str4 = new String(byteArr);
        System.out.println(str4);

        String str5 = new String(str3);
        System.out.println(str5);

        //Meerut
        System.out.println(str.length());
        System.out.println(str.charAt(3));

        System.out.println(str.substring(2,5));
        System.out.println(str.toUpperCase());
        System.out.println(str.toLowerCase());

        String str6 = new String(str);
        System.out.println(str.equals(str6));
        System.out.println(str==str6);
        str6="meeRut";
        System.out.println(str.equalsIgnoreCase(str6));

        System.out.println(str.indexOf("ru")); //3
        System.out.println(str.lastIndexOf("e")); //2

        System.out.println(str.startsWith("Mee"));
        System.out.println(str.endsWith("rut"));
        System.out.println(str.endsWith("rutt"));

        System.out.println("  Amol  ".trim());
        System.out.println("Sharma".replace('a','m'));

    }
}

```

Object Oriented Programming with Java

Package

*“A package is a **collection of related Java elements** such as classes, interfaces etc.”*

Uses of Packages in Java

1. Namespace Management

Packages are used to *divide the class name space in to compartments*. A class name must not be repeated in a compartment but it may be repeated/ reused in other compartments.

Naming conflict of classes may be resolved by prefixing the class name with a package name.

Example:

pack1.Rectangle and pack2.Rectangle are two distinct classes. Although they share the same class name Rectangle, but they belong to two different packages: pack1 and pack2.

Object Oriented Programming with Java

Package

Uses of Packages in Java

2. Access Control

We can define classes inside a package that are not accessible by code outside that package. We can also define class members that are exposed only to other members of the same package.

3. Reuse and Distribution

Packages group related reusable classes and interfaces together. This collection may be distributed as *Java Archive (JAR)* files for easy sharing and reuse.

Object Oriented Programming with Java

Package

Package Name & Directory Structure

In Java, package names are mapped to the directory structure used to store the class files.

Example:

If a class Rectangle belongs to the package p1.pack1.project1, it is stored as: *p1\pack1\project1\Rectangle.class* in the current working directory.

Packages with Common Prefixes

If we have package names like *com.school* and *com.school.exam*. Although their names seem related, they are two different packages. They just share part of the directory path. The classes in the package *com.school* are stored in *com\school* and the classes in the package *com.school.exam* are stored in *com\school\exam*.

Object Oriented Programming with Java

Package

Naming Convention for Packages

In Java, packages are named using a naming convention to avoid conflicts between different organizations and projects.

- Use Your Organization's Internet Domain Name in Reverse

Example, if your company's domain is **example.com**, you should start your package name with **com.example**.

Example:

com.example.myapp

- Use all Lowercase Letters, package names are written in all small letters.
- Choose meaningful names after the reversed domain, use descriptive names of the project or module.

Example:

com.example.shopping.cart

Object Oriented Programming with Java

Package

Defining a Package

A package in Java is defined *using the package keyword as the first statement* in the source file. All the classes and interfaces in that file become part of the specified package.

Example:

```
package mypack;
```

Note:

- If no package is defined, the class belongs to the **default package (unnamed)**.
- **Multiple files** can belong to the **same package** by using the same package statement.
- **Directory name** must match the **package name** exactly (Java is case-sensitive). For `package a.b.c;`, compiled .class files should be stored in the folders on the path `a\b\c`.

Object Oriented Programming with Java

Package

Compiling Java Classes into Package Directories

Two Ways to Organize .class Files in Packages:

1. *Manual Method*

Compile the Java file normally

```
javac MyClass.java
```

Create the required directory structure matching the package name.

For *package mypack*; create *mypack/* directory.

Move the .class file manually to mypack directory.

2. *Using -d Option (Recommended)*

Compile and automatically create the package directory structure.

```
javac -d classes MyClass.java
```

If MyClass.java contains *package mypack*;

The file is stored as:

```
classes/mypack/MyClass.class
```

Object Oriented Programming with Java

Package

CLASSPATH Setting for Packages

To run Java programs that use user-defined packages, the Java runtime needs to know where to find those packages.

Three Ways to find the packages:

1. Default: Current Working Directory

By default, Java looks in the current working directory.
If your package is in a subdirectory of the current directory, Java will find it automatically.

2. Using the CLASSPATH Environment Variable

You can define the path to your package directory using the system's CLASSPATH environment variable.

Important: The CLASSPATH should contain the path up to the package folder, not the package name itself.

Object Oriented Programming with Java

Package

CLASSPATH Setting for Packages

Three Ways to find the packages:

2. Using the CLASSPATH Environment Variable

Example:

If the package is in D:\docs\JavaPrograms\Java\classes\mypack then set

CLASSPATH = D:\docs\JavaPrograms\Java\classes

3. Using '-classpath' or '-cp' Option

While compiling and running, you can specify the class path using the -classpath or -cp option.

Example:

java -classpath D:\docs\JavaPrograms\Java\classes mypack.MyClass

```
package com.example.myproject;
class MyClass
{
    int attr;

    void setAttr(int attr)
    {
        this.attr = attr;
    }

    int getAttr()
    {
        return attr;
    }
}
class MyClassMain
{
    public static void main(String args[])
    {
        MyClass obj = new MyClass();
        obj.setAttr(34);
        System.out.println(obj.getAttr());
    }
}
```

Object Oriented Programming with Java

Package

Access Control

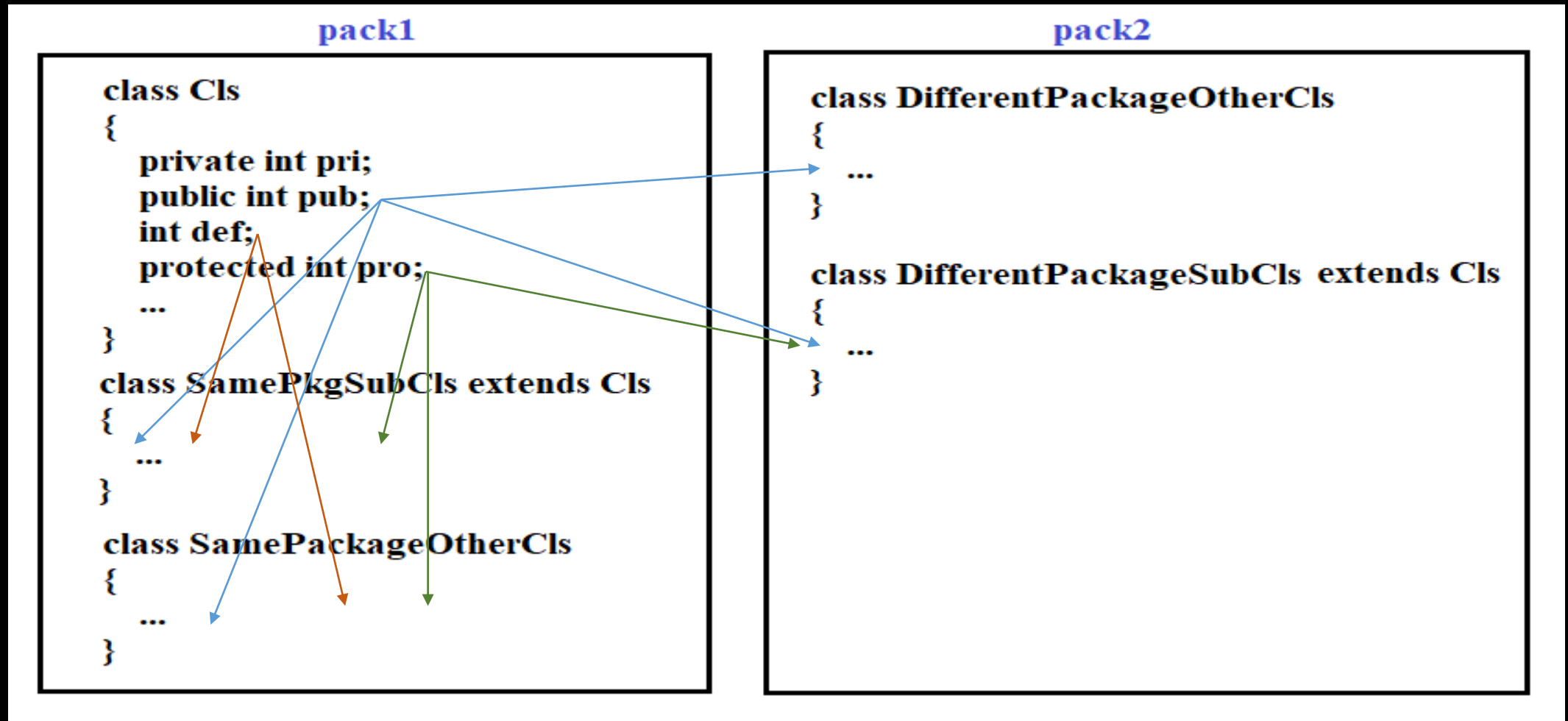
- **public** members of a class can be accessed from different classes and different packages.
- **private** members cannot be seen outside of its class.
- When a member has **no access specifier**, it is visible to all the other classes in the same package. This is the **default access**.
- **protected** members of a class can be accessed by any other class of the package as well as by the subclasses in other packages.

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

Object Oriented Programming with Java

Package

Access Control



Object Oriented Programming with Java

Package

Access Control

A class or an interface has only two possible access levels: default and public.

- When a class/ interface is declared as **public**, it is **accessible outside its package**.
When a class is public, it must be the only public class declared in the file, and the file must have the same name as the class.
- If a class/ interface has **default** access, then it can only be **accessed by other code within its same package**.

Object Oriented Programming with Java

Package

Importing Packages

*In Java, classes and interfaces from packages must be imported before they can be used in a program (unless they belong to the same package as the current class). This is done using the **import statement**.*

- import statements must appear after the package statement (if any) and before any class definitions.
- Classes in the same package can be accessed without import.
- Import can be used to access built-in classes from built-in packages like java.util, java.io, etc.
- The **java.lang** package is automatically imported in every Java program and contains fundamental classes such as String, System etc.

Example:

```
import java.util.Scanner; // Imports only Scanner class
import java.util.*; //Imports all classes in java.util)

import mypackage.MyClass; // User-defined package and class
import mypackage.*;      // Imports all classes from mypackage
```

Object Oriented Programming with Java

Package

Static Import

Static import allows members (fields and methods) defined as static in other classes to be used without class qualification.

It is useful when we frequently use static members like constants or utility methods in the code.

Example:

```
import static java.lang.Math.PI;

// Imports all static members like sqrt, pow, PI, etc.
import static java.lang.Math.*;

public class Test {
    public static void main(String[] args) {
        System.out.println(PI);    // No need to write Math.PI

        // No need to write Math.sqrt(16)
        System.out.println(sqrt(16));
    }
}
```

Object Oriented Programming with Java

Package

Making JAR Files for Library Packages

*A **JAR (Java Archive)** file bundles together multiple .class files, images, and other resources into a single compressed file for distribution and reuse.*

Creating a JAR File

```
jar cf MyLib.jar mypack
```

c → Create a new JAR

f → Specify JAR file name

mypack → Folder containing .class files (package)

Using the JAR in Another Program

Compile source file (.java file) using classes available in the JAR file specified in the classpath.

```
javac -cp utility.jar Main.java
```

Run the compiled class, setting the classpath to include the current directory (.) and the JAR file.

```
java -cp .;utility.jar Main
```

Object Oriented Programming with Java

Comments

Comments are used to make code more understandable. They are ignored by the compiler and do not affect the execution of the program.

Types of Comments:

Single-line Comment (//):

It is used for brief explanations or notes spanning only a single line.

Example:

```
// This is a single-line comment  
int x = 10; // declaring a variable
```

Multi-line Comment (/ ... */)*

It is used for longer descriptions spanning multiple lines or to temporarily disable code.

Example:

```
/* This is a  
multi-line comment */
```

Object Oriented Programming with Java

Command Line Arguments

These are the arguments (values) passed to the main() method during program execution from the command line.

- All command line arguments are stored as String objects in the args[] array.
- args.length gives the number of arguments.
- Arguments are indexed from 0.

Example:

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("First Argument: " + args[0]);  
        System.out.println("Second Argument: " + args[1]);  
    }  
}
```

Executing program with Command Line Arguments

> **java Example Hello Java**

```
package com.example.myproject;
public class MyClass
{
    int attr;

    public void setAttr(int attr)
    {
        this.attr = attr;
    }

    public int getAttr()
    {
        return attr;
    }
}
```

```
import com.example.myproject.MyClass;
//import com.example.myproject.*;

class MyClassMain
{
    public static void main(String args[])
    {
        //fully qualified name - com.example.myproject.MyClass -
feasible        MyClass obj = new MyClass();
                obj.setAttr(34);
                System.out.println(obj.getAttr());
    }
}
```

```
class CmdLineArgs
{
    public static void main(String[] mainArgs)
    {
        System.out.println(mainArgs[0]);
        System.out.println(mainArgs[1]);
        System.out.println(mainArgs[2]);
        System.out.println(mainArgs[3]);
    }
}
```